

Teste 1 (Avaliação das Aulas 1 e 2)

Objetivo: Criar um sistema de controle de iluminação robusto e eficiente, aplicando conceitos de memória e lógica não-bloqueante.

- **Problema Proposto:** "Crie um 'Dimmer Digital'."
 1. **Hardware (Botões):** Você terá dois botões: um para 'Aumentar' e outro para 'Diminuir'.
 2. **Hardware (LED):** Você terá um LED conectado a uma saída PWM (os pinos com o símbolo ~) para controlar o brilho.
 3. **Estrutura de Dados:** Crie uma `struct` chamada `ControleLED` que agrupa duas informações: o número do pino do LED e o valor do brilho atual (0 a 255).
 4. **Lógica:** No `loop()`, você deve ler os botões. **Proibido usar `delay()`.**
 5. **Ação:** Ao clicar em 'Aumentar', o brilho sobe (ex: +10). Ao clicar em 'Diminuir', o brilho desce (ex: -10). O brilho deve respeitar os limites **0 e 255**.
 6. **Função com Referência:** Crie uma função `void atualizaLED(ControleLED& led)` que:
 - Recebe a sua `struct` por **referência** (&).
 - Aplica o brilho no pino usando `analogWrite()`.
 7. **Bônus:** Use `Serial.println(F("..."))` para imprimir o brilho atual no monitor serial, mas **apenas quando o valor mudar** (para não poluir a tela).
- **Critérios de Avaliação:** Lógica não-bloqueante (`millis`/leitura de botão), uso correto de `structs`, passagem por referência (&), uso de PWM (`analogWrite`) e economia de memória (`F()`).

Dicas de Sobrevivência (Leia antes de começar!)

1. O Problema da Leitura (Borda e Ruído): O Arduino é milhões de vezes mais rápido que o seu dedo. Isso cria dois problemas que você precisa tratar no seu `if`:

- **Deteção de Borda (State Change):** Se você verificar apenas se o botão *está* pressionado (`LOW`), o `loop()` vai somar +10 milhares de vezes enquanto você ainda está encostando o dedo.
 - *Correção:* Verifique se o botão ficou `LOW` **agora**, mas estava `HIGH` **antes**.
- **Debounce (Ruído Elétrico):** No instante do contato metálico, o sinal gera um "ruído" elétrico rápido (oscila entre 0 e 1) antes de estabilizar. O Arduino lê isso como múltiplos cliques.
 - *Correção:* Implemente um "**Cooldown**" com `millis()`. Após detectar um clique válido, ignore qualquer novo sinal daquele botão pelos próximos **200ms**.

2. O "Mundo Invertido" do INPUT_PULLUP: Ao configurar os pinos no `setup()`, use a sintaxe correta: `pinMode(PINO, INPUT_PULLUP);`.

- **Por que usar?** Isso ativa um resistor interno do Arduino que "puxa" a voltagem para 5V quando o botão está solto, evitando leituras aleatórias.
- **A Consequência (Lógica Invertida):**
 - **Botão SOLTO (Repouso):** O resistor interno mantém o pino em 5V. O Arduino lê **HIGH (1)**.
 - **Botão APERTADO (Ação):** Você conecta o pino ao GND (Terra). A voltagem cai para 0V. O Arduino lê **LOW (0)**.
 - **Resumo:** No seu `if`, você deve verificar se o botão é igual a `LOW` para saber se ele foi apertado!

3. A Matemática Perigosa (O Segredo do `int16_t`):

A variável de brilho na struct é `uint8_t` (0 a 255).

- **O Perigo:** Se o brilho for 0 e você fizer `brilho - 10`, a matemática de 8 bits não aceita negativos e dá a volta para 246. O LED acende forte em vez de apagar!
- **A Solução:** Dentro do loop, crie uma **variável temporária** do tipo `int16_t` (que aceita negativos e números maiores que 255).
 1. Copie o valor da struct para essa variável temporária.
 2. Faça a soma/subtração nela.
 3. Verifique se estourou os limites (`if < 0` zera, `if > 255` trava em 255).
 4. Só depois de corrigir, salve de volta na struct.

4. Lembre-se do PWM (0-255):

O `analogWrite` usa uma resolução de 8 bits.

- Ele **só funciona** em pinos marcados com ~ (ex: 3, 5, 6, 9, 10, 11).
- O valor mínimo é 0 (apagado) e o máximo é 255 (brilho total). Não tente escrever 300 ou -50 direto no pino.

Desafios Extras (Para quem quer ir além)

Terminou e funcionou? Tente implementar estas melhorias:

1. **Botões de Atalho (Máximo/Mínimo):** Adicione mais dois botões ao circuito. Um botão "MAX" que leva o brilho instantaneamente para 255, e um botão "OFF" que leva instantaneamente para 0. Use a mesma lógica de debounce e struct.
2. **Interface Humana (Porcentagem):** O usuário comum não sabe o que é "brilho 127". Modifique seu `Serial.println` para exibir o brilho em **porcentagem (0% a 100%)** em vez do valor cru (0 a 255).
 - *Dica de Matemática:* Converta da escala 0-255 para 0-100.